

Utilização de Padrões para Otimizar a Automação de Testes Funcionais de Software

Rafael Braga de Oliveira^{1,3}, Francisco Nauber Bernardo Góis³,
Jerffeson Teixeira de Souza², Pedro Porfírio Muniz Farias¹

¹Universidade de Fortaleza (UNIFOR)
Av. Washington Soares, 1321 – Fortaleza – CE – Brasil

²Universidade Estadual do Ceará (UECE)
Av. Paranjana, 1700 - Fortaleza – CE - Brasil

³Serviço Federal de Processamento de Dados (SERPRO)
Av. Pontes Vieira, 832 - Fortaleza – CE - Brasil

{rafael.oliveira, francisco.gois}@serpro.gov.br,
jeff@larc.es.uece.br, porfírio@unifor.br

Resumo. A automação de testes funcionais tem se tornado um evidente atrativo para equipes de desenvolvimento de software. Tal fato se deve principalmente à grande redução de custo observada a médio e a longo prazos com o uso desta prática. Este artigo propõe a aplicação de padrões para otimizar a automação de testes funcionais de software, introduzindo benefícios como o aumento da reusabilidade e da manutenibilidade de scripts de teste, e a facilidade de inclusão de novos casos de teste. O uso de padrões na automação de testes funcionais representa uma aplicação inovadora de padrões e um diferencial em relação às técnicas de automação de testes funcionais citadas na literatura.

Palavras-chave: automação de testes; testes funcionais; teste de *software*, padrões de projeto.

Abstract: The functional testing automation has become a real interest to software development teams, mainly because of the great cost reduction observed on medium and long terms with the use of this practice. This article proposes the application of patterns to optimize the software functional testing automation, introducing benefits as the increase of reusability and maintainability of tests scripts, and the facility of including new test cases. The use of patterns on functional testing automation represents an innovative application of patterns and a improvement in relation to the techniques of the functional testing automation mentioned on the literature.

Keywords: testing automation; functional testing; software testing, regression testing, design patterns.

1. Introdução

Para minimizar o custo e proporcionar maior qualidade no desenvolvimento de *software*, inúmeros estudos ressaltam a importância de um processo de teste efetivo [1, 2, 3, 4]. Desta forma, quanto mais eficiente e mais eficaz for o teste, menor será o custo dos reparos e maior será a qualidade do produto.

Evidentemente, quanto mais tarde um defeito é encontrado maior é o custo de sua correção, podendo, em casos extremos, causar danos irreparáveis. Neste contexto, a automação se tornou uma excelente alternativa para reduzir o esforço requerido à realização de testes funcionais de modo adequado. Segundo FEWSTER [2], testes

manuais que levariam horas para serem concluídos, podem ser executados em minutos, quando automatizados.

Este processo de automação é baseado na criação e na execução de *scripts* de teste através de ferramentas *Record and Playback*, também conhecidas como ferramentas *Capture and Replay*. Algumas destas ferramentas permitem a criação destes *scripts* na forma de programas Java, que representam as ações de um usuário ao interagir com a interface do sistema.

Tais programas podem ser agrupados em *frameworks* que são consideravelmente beneficiados aplicando-se boas técnicas de programação. As técnicas *Data-driven* [2, 11] e *Keyword-driven* [2, 12] têm sido aplicadas nestes *frameworks* [11].

Neste contexto, propomos um *framework* denominado FuncTest, que, além das duas técnicas citadas, faz uso de padrões [6, 7, 8] para otimizar o projeto de automação de testes funcionais. Foram utilizados a arquitetura MVC [6], o padrão DAO (*Data Access Object*) [8] e, em duas situações, o padrão *Factory Method* [5].

Com a utilização da arquitetura MVC, associamos cada passo do caso de testes, através de uma tabela, ao *script* a ser executado neste passo. Assim, podemos, por exemplo, para o mesmo passo do caso de teste, com os mesmos dados, executar *scripts* diferentes, cada um dos quais com uma sequência de ações diferentes sobre a interface, mas que deveriam redundar no mesmo resultado.

O padrão DAO foi utilizado para fornecer independência na persistência dos dados associados ao código do *script*, permitindo assim que os dados de teste persistam em bases de dados distintas. A geração dos objetos DAO foi implementada através de uma aplicação do padrão *Factory Method*. O padrão *Factory Method* também foi utilizado para a criação dos *scripts* de teste.

Nossa proposta se encontra em uso por especialistas de teste numa grande empresa estatal de desenvolvimento de *software*. O processo de desenvolvimento padrão utilizado é aderente ao nível 2 do CMM. Uma versão preliminar do *framework* foi premiada internamente, em nível nacional, entre os melhores projetos de 2006. A versão atual é uma evolução que além de utilizar os padrões permite configurar o *framework* através de arquivos XML.

Na próxima seção, discutiremos sobre a automação de testes funcionais e algumas técnicas associadas, evidenciando-se problemas que serão tratados em seções subsequentes através do uso de padrões. Na seção 3 abordaremos os padrões utilizados. Na seção 4, apresentaremos o *framework* proposto. Por fim, na seção 5, serão apresentados a conclusão e os trabalhos futuros.

2. Automação de Testes Funcionais

A automação de testes funcionais é realizada através da criação e da execução de *scripts* de teste a partir de ferramentas Record and Playback. Estas ferramentas permitem que os *scripts* sejam programados ou simplesmente gravados durante a realização de um teste manual, podendo ser alterados programaticamente.

Tais ferramentas permitem a inclusão de pontos de verificação durante a execução de cada *script*. Isto possibilita verificar se o comportamento da aplicação está de acordo com o esperado. De outra forma, não haveria teste, e sim apenas uma robotização da interação sobre a interface de usuário. Com base nestes pontos de verificação, é gerado um relatório final apresentando o resultado do teste.

Embora estas ferramentas contribuam para a realização de testes mais efetivos, o sucesso de um teste não é garantido simplesmente através de sua automação, pois esta poderá implementar um caso de teste mal elaborado ou despender um custo inviável para a sua construção. Além disso, para justificar este custo, a decisão por se automatizar um teste deve pressupor que o mesmo será realizado recorrentemente. Normalmente, a automação de testes funcionais é utilizada para a realização de testes de regressão.

Um teste de regressão corresponde à re-execução de um teste outrora realizado com sucesso sob um conjunto de funcionalidades do sistema. Isto é feito após alterações sofridas pelo sistema, a fim de assegurar que o funcionamento do mesmo permanece de acordo com o especificado.

O esforço para a construção e manutenção de um teste funcional automatizado se justifica com a economia gerada após sucessivas execuções dos *scripts* criados. Certamente, o custo destas execuções é significativamente menor do que o necessário à realização de testes manuais equivalentes.

Utilizamos em nosso estudo a ferramenta *Rational XDE Tester*, a qual utiliza uma instância do Eclipse, um ambiente de desenvolvimento integrado (IDE - *Integrated Development Environment*). Nela, cada *script* é gravado ou programado como uma classe Java e conta com um mapa de objetos gerado durante a gravação dos *scripts* ou configurado pelo analista responsável pela automação. Este mapa registra uma pontuação para cada objeto do *browser*, baseado em suas características. Isto permite que a ferramenta identifique os objetos durante a execução de um *script*.

Os *scripts* gerados através das ferramentas *Record and Playback* para a execução dos testes de regressão podem ser extensos, ilegíveis, difíceis de serem mantidos e reutilizados, bem como apresentar em sua estrutura dados “*hard-coded*” e comandos desnecessários.

Normalmente os *scripts* gerados pelas ferramentas *Record and Playback* deverão ser alterados ou, até mesmo, ser inteiramente criados programaticamente pelo analista. Por exemplo, o analista deve excluir comandos desnecessários inseridos pela ferramenta e utilizar boas práticas de programação possíveis, como a inclusão de comentários para esclarecer a lógica do código.

Neste contexto, é importante entender que o projeto de automação de testes deve ser tratado como qualquer outro projeto de desenvolvimento de *software*. Portanto, o esforço de criação e manutenção de um teste automatizado pode ser significativamente reduzido através do uso apropriado de padrões.

Apresentamos a seguir alguns requisitos identificados para aperfeiçoar o projeto de automação:

- *scripts* coesos: é importante que os *scripts* sejam bastante específicos. Isto possibilitará melhor reuso;
- *scripts* independentes: a chamada de um *script* não deve ser feita a partir de um outro, pois assim estaremos criando *scripts* pouco reutilizáveis;
- ausência de dados “*hard-coded*”: os dados informados pelo usuário durante a gravação dos *scripts* não deveriam ser armazenados explicitamente no código do *script*. Assim, estaríamos produzindo *scripts* difíceis de serem mantidos ou reutilizados;
- controle de erros independente: o fluxo de tratamento de exceções deve ser independente do fluxo normal de execução dos *scripts*, facilitando a manutenção e o reuso do *script*;
- múltiplas bases de testes: o projeto de automação pode se tornar mais flexível quando permite obter dados de diversas bases.

Uma das técnicas amplamente utilizadas e essenciais para tornar a automação mais reutilizável é a técnica *Data-driven*. Esta técnica, propõe a independência entre o código do *script* e a massa de dados utilizada durante o teste, evitando dados “*hard-coded*”. Para aplicá-la, o analista deverá extrair os dados de entrada do corpo do *script* e armazená-los em arquivo independente. Isto permite um mesmo *script* seja utilizado para vários conjuntos de dados de entrada.

A técnica *Keyword-driven* propõe a modularização dos *scripts*, de maneira que cada módulo seja representado por uma *keyword*. As *keywords* tanto podem representar eventos simples, como um clique num botão, a serem aplicados na interface do sistema [12] como módulos mais genéricos, por exemplo, o processamento de uma opção do sistema que envolva navegar em várias telas. Independente da complexidade do módulo, as *keywords* representam ações, de maior ou menor complexidade, sobre a interface do sistema.

Um Caso de teste é uma seqüência de *steps*. O nome do *step* pode ser considerado uma *keyword*, conforme preconiza a técnica *Keyword-driven*. Para cada *step* de um caso de teste, é executado um *script* associado com seus respectivos dados de teste.

Na próxima seção, serão indicados os padrões utilizados na formulação do *framework* proposto.

3. Padrões Utilizados

O *framework* FuncTest faz uso dos padrões *Factory Method* e DAO. Nesta seção, descreveremos brevemente tais padrões para, em seguida, evidenciarmos suas aplicações no *framework*.

3.1 Padrão *Factory Method*

O padrão *Factory Method* [5], também conhecido como *Virtual Constructor*, é especificado utilizando dois níveis: um nível abstrato e um nível concreto.

No nível abstrato, especificaremos a utilização de um construtor virtual, o *Factory Method*. Assim, lidaremos, neste nível, com a construção virtual de objetos, ainda sem antecipar a classe dos objetos que serão criados. No nível concreto, criaremos classes que estendem as classes abstratas implementando apropriadamente o construtor virtual.

No diagrama da Figura 1, temos, no nível abstrato, um objeto da classe abstrata *Creator* que deseja criar um objeto da classe abstrata *Product*. Como ambas as classes são abstratas não é possível, neste nível, antecipar que classe concreta será utilizada.

Especifica-se então o método abstrato *FactoryMethod()* que prevê a devolução do produto desejado como resultado. Este método funciona como um construtor virtual.

No nível concreto, temos as classes *ConcreteProduct*, subclasse da classe *Product*, e a classe *ConcreteCreator*, subclasse da classe *Creator*.

A classe *ConcreteCreator* implementa o construtor virtual através do método concreto *FactoryMethod()*, que devolve como resultado um objeto da Classe *ConcreteProduct*.

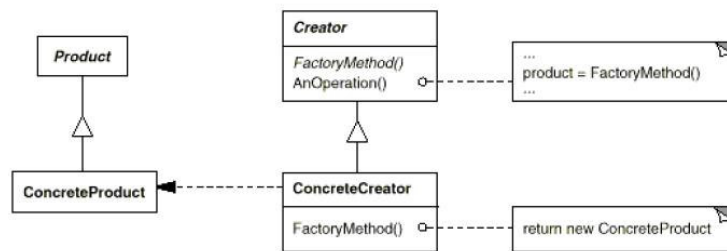


Figura 1 - Diagrama de Classes do Padrão *Factory Method* [5]

3.2 Padrão DAO

O padrão DAO encapsula o modo de acesso aos dados das classes de negócio, permitindo a utilização de fontes de dados distintas e tornando a obtenção dos dados transparente. As classes de negócio terão acesso apenas a objetos e não mais a código SQL.

Este padrão elimina a necessidade de conhecimento prévio da fonte de dados e dos tipos de *drivers* e *interfaces* utilizados para acesso à persistência.

No diagrama da Figura 2, é apresentado o relacionamento entre os participantes deste padrão.

A classe *BusinessObject* é a classe de negócio que utilizará o padrão através de chamada à classe *DataAccessObject*. A classe *DataAccessObject* implementa a forma de acesso aos dados, retornando um objeto da classe *TransferObject*. A classe *DataSource* representa a forma de acesso que é encapsulada pela classe *DataAccessObject* e normalmente refere-se a uma classe que implementa a interface *JDBC*, no caso de um sistema J2EE.

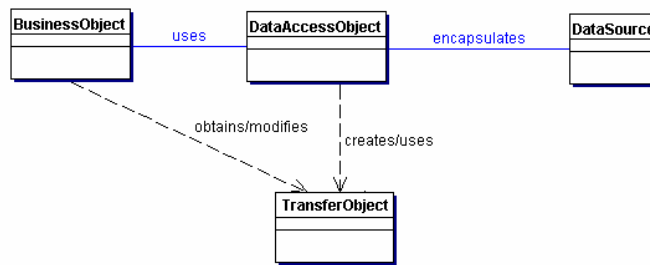


Figura 2 – Diagrama de Classes do Padrão DAO [8]

4. O Framework FuncTest

O *framework* FuncTest está sendo utilizado dentro de um processo de desenvolvimento aderente ao CMM nível 2 com o objetivo de melhorar a produtividade na automatização de testes funcionais realizados com a utilização de ferramentas Record and Playback.

O FuncTest foi desenvolvido com a utilização da ferramenta *Rational XDE Tester* mas poderá ser utilizado em ferramentas similares que também gerem código Java.

A principal característica do *framework* é permitir que os *scripts* contenham somente ações de interação com interface de usuário. Desta forma, outras responsabilidades, como chamada a outros *scripts*, acesso a dados e controle de erros, são desvinculadas dos *scripts*, fortalecendo o reuso e a manutenibilidade dos mesmos.

Nossa proposta prevê a criação de procedimentos de teste que contem um número arbitrário de casos de teste. Cada caso de teste é composto de vários *steps*. Conforme preconiza a técnica *Keyword-driven*, cada passo do caso de teste contem uma *keyword*, o nome do *step*.

Utilizando a arquitetura MVC, um controlador associa cada *keyword* ao *script* que deverá ser executado. Como propõe a técnica *Data-driven*, os dados de teste são mantidos desvinculados dos *scripts*. Assim eles poderão ser mais facilmente mantidos e reutilizados.

A arquitetura adotada no FuncTest permite o controle independente de chamada dos *scripts*. Desta forma, evitamos que um *script* chame outro *script* eliminando o acoplamento entre eles.

O *framework* permite que os erros, que interromperiam a execução de um procedimento de testes, possam ser manipulados por *scripts* especificamente construídos com esta finalidade.

Descreveremos na seção 4.1 os pacotes do *framework* distribuídos segundo a arquitetura MVC.

O *framework* é configurado através de um conjunto de arquivos XML, assim, só é preciso manipular código java na elaboração dos *scripts*. Na seção 4.2 descreveremos os arquivos de configuração do *framework*.

O funcionamento do FuncTest será abordado na seção 4.3. através de um diagrama de seqüência e na seção 4.4 será traçado uma correspondência entre as classes preconizadas nos padrões e aquelas implementadas no *framework*.

4.1 Pacotes do *Framework*

O *framework* é formado basicamente pelos pacotes Model, View e Controller, e um conjunto de arquivos de configuração. O diagrama de classes a seguir mostra uma visão resumida de seus pacotes, contemplando as suas principais classes.

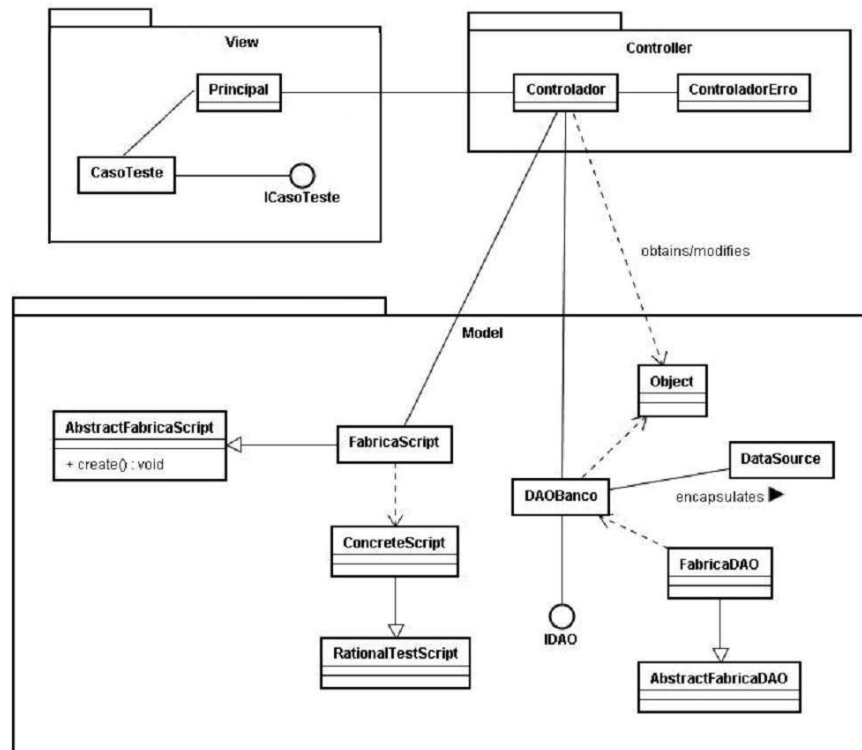


Figura 3 – Diagrama de Classes Resumido do FuncTest

O pacote Model é formado pelas classes, FabricaScript, FabricaDAO, Erro, Step, DataSource, Util, e pela interface IDAO.

O pacote View é constituído pela a classe Principal, pela Classe CasoTeste e pela interface ICasoTeste.

O pacote Controller é formados pelas classes Controlador e ControladorErro.

4.2 Arquivos de Configuração

Os arquivos de configuração usados pelo FuncTest seguem o formato XML. São eles: Procedimento.xml, <NomeDoCasoDeTeste>.xml, Scripts.xml, Erros.xml, Banco.xml e DataSource.xml.

Os dois primeiros arquivos: Procedimento.xml, <NomeDoCasoDeTeste>.xml persistem os dados da camada de visão.

Os arquivos Scripts.xml e Erros.xml, estão associados a camada de controle.

Scripts.xml representa a tabela onde o Controlador associa cada *step* do caso de testes ao *script* que será executado

Erros.xml representa a tabela que registra os *scripts* de tratamento de erro a serem chamados pelo Controlador de erros.

A camada de modelo é constituída por um conjunto de *scripts* (classes java) e seus respectivos dados.

Para cada *step* o controlador utiliza um padrão *Factory Method* para recuperar o *script* apropriado e outro *Factory Method* para recuperar o *datasource* apropriado.

O arquivo DataSource.xml indica as informações associadas a cada *datasource*. Enquanto o arquivo Banco.xml informa, para cada SGBD, uma classe correspondente no padrão DAO

A estrutura de cada um desses arquivos será detalhada a seguir.

Procedimento.xml

É formado pelo conjunto de *tags* denominadas CasoTeste. Cada uma destas *tags* possuirá duas seções:

Nome: representa o nome do caso de teste;

Arquivo: representa o nome do arquivo XML com a descrição do caso de teste.

```
TEMPLATE PROCEDIMENTO.XML

<CasoTeste>
  <Nome>NomeDoCasoDeTeste1</Nome>
  <Arquivo>NomeDoCasoDeTeste.xml</Arquivo>
</CasoTeste>
<CasoTeste>
  <Nome>NomeDoCasoDeTeste2</Nome>
  <Arquivo>NomeDoCasoDeTeste.xml</Arquivo>
</CasoTeste>
```

Quadro 1 - Template para criação do arquivo Procedimento.xml

<NomeDoCasoDeTeste>.xml

Para cada caso de teste, deve ser criado um arquivo correspondente. Estes arquivos serão formados pelo conjunto de *tags* denominadas de Step. Cada *tag* Step possuirá quatro elementos:

Nome: representa o nome do *step*;

Tipo: dependendo de seu valor (“*Script*” ou “*DataSource*”), a Fábrica retornará respectivamente um *script* predefinido ou a escolha do *script* será baseada nos dados de teste do *datasource* correspondente;

DataSource: esta *tag* é opcional. Ela define uma fonte de dados (*datasource*) para os dados de teste;

Numero: esta *tag* é opcional. Corresponde ao número de vezes que o *script* será executado.

```
TEMPLATE <NOME DO CASO DE TESTE>.XML

<step>
  <Nome>Step01</Nome>
  <Tipo>DataSource</Tipo>
  <DataSource>NomeDataSource</DataSource>
  <Numero>000</Numero>
</step>
<step>
  <Nome>Step02</Nome>
  <Tipo>Script</Tipo>
</step>
```

Quadro 2 - Template para criação dos arquivos <NomeDoCasoDeTeste>.xml

Scripts.xml

Contém uma *tag* indicando o *script* correspondente a cada *step* dos casos de teste.

```
TEMPLATE SCRIPTS.XML

<scripts>
  <Step01>Script01</Step01>
  <Step02>Script02</Step02>
</scripts>
```

Quadro 3 - Template para criação dos arquivos Scripts.xml

Banco.xml

Para cada banco de dados, contém uma *tag* <NomeBanco> que indica a classe DAO correspondente.

```
TEMPLATE BANCO.XML

<bancos>
  <ORACLE>DAOOracle</ORACLE>
  <SQLSERVER>DAOSqlServer</SQLSERVER>
  <DB2>DAODB2</DB2>
```

```

<ACCESS>DAOAccess</ACCESS>
<INTERBASE>DAOInterbase</INTERBASE>
<MYSQL>DAOMySQL</MYSQL>
<SYBASE>DAOSybase</SYBASE>
</bancos>

```

Quadro 4 - Template para criação dos arquivos Banco.xml

DataSource.xml

Para cada *step* cuja *tag* <Tipo> possua o valor “*Datasource*”, deverá ser descrita a respectiva fonte de dados. O arquivo Datasource.xml é um repositório que contem as descrições das diversas fontes de dados utilizadas. No exemplo do quadro 5 temos um *datasource* denominado <NomeDataSource01>. As fontes de dados são descritas por *tags* que representam a *string* de conexão, o nome do banco, a tabela de dados, o *login* e a senha do *datasource*, o nome do *driver*, o número de colunas a ser selecionado na tabela e o nome da coluna para a ordenação da consulta.

```

TEMPLATE DATASOURCE.XML

<DadosDeTeste>
  <NomeDataSource01>
    <Conexao>StringConexao</Conexao>
    <Tabela>NomeTabela</Tabela>
    <Login>Login</Login>
    <Driver>NomeDriver</Driver>
    <Senha>Senha</Senha>
    <Colunas>NumeroColunasTabela</Colunas>
    <Banco>Banco01</Banco>
    <Ordenado>ColunaParaOrdenacao</Ordenado>
  </NomeDataSource01>
</DadosDeTeste>

```

Quadro 5 - Template para criação dos arquivos DataSource.xml

Erros.xml

É formado por um conjunto de *tags* cada uma associando uma Exceção ao *script* executado quando de sua ocorrência.

```

TEMPLATE ERROS.XML

<erros>
  <Excecao1>Nome.do.Script.que.trata.a.Excecao1 </Excecao1>
  <Excecao2>Nome.do.Script.que.trata.a.Excecao2 </Excecao2>
</erros>

```

Quadro 6 - Template para criação dos arquivos Erros.xml

Para o funcionamento do *framework*, que será detalhado a seguir, a apropriada configuração dos arquivos XML mencionados permitirá a associação dos passos dos casos de teste aos *scripts* a serem executados com os respectivos dos dados de teste.

4.3. Funcionamento do Framework

O funcionamento do FuncTest será apresentado com base no Diagrama de Seqüência a seguir.

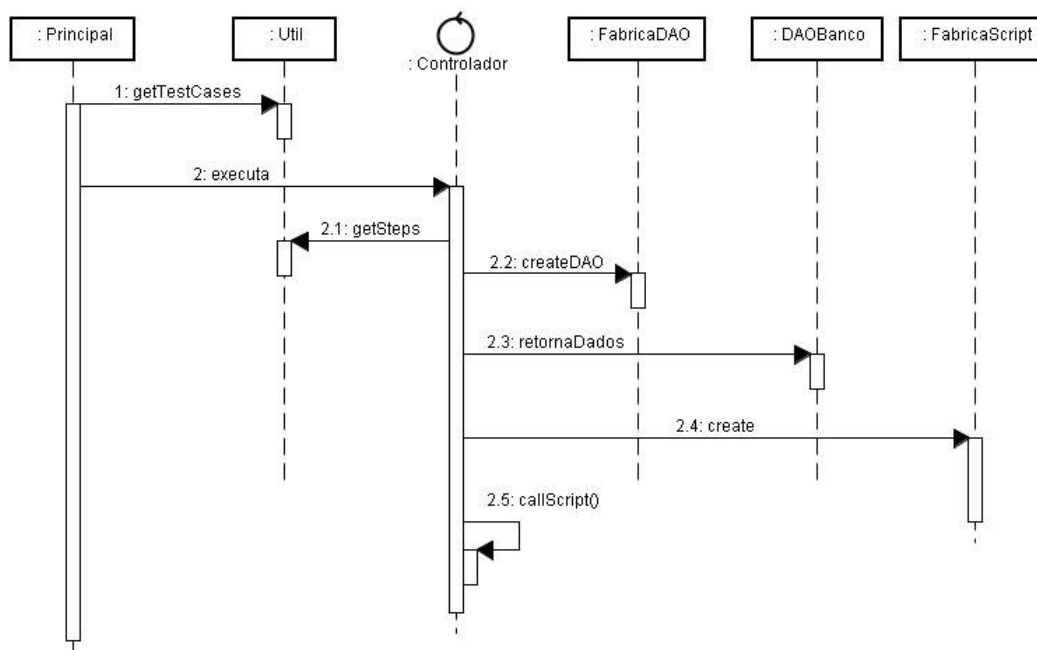


Figura 4 – Diagrama de Seqüência Simplificado do *framework* FuncTest

A classe Principal é responsável por iniciar o processo de automação. Ela irá solicitar à classe Util, responsável pela deserialização de todos os arquivos XML, os casos de teste contidos no arquivo Procedimento.xml.

Para cada caso de teste, a classe Principal irá criar um objeto casoTeste e solicitar à classe Controlador a sua execução. Cada objeto casoTeste é uma instância da classe CasoTeste, a qual implementa a interface ICasoTeste.

A classe Controlador, após receber cada objeto casoTeste, irá obter os *steps* do caso de teste correspondente contidos no arquivo <NomeDoCasoDeTeste>.xml. Para cada objeto *step* deverão ser criados o *script* e, se o tipo do *step* for “*datasource*”, deverão ser recuperados os dados de teste.

Para recuperação dos dados de teste é consultado o arquivo Datasource.xml. Neste arquivo será identificado o banco de dados a ser utilizado e então será solicitada à classe FabricaDAO a criação do respectivo objeto DAO.

A classe FabricaDao solicita a criação do objeto DAO que será uma instância da classe definida no arquivo Banco.xml. O objeto DAO será utilizado para recuperar os dados para a realização do teste.

A criação do objeto *script* é solicitada a classe *FabricaScript* que consulta a tabela contida no arquivo *Script.xml* para identificar o *script* associado ao *step*.

Uma vez criados os dados de teste e o *script*, o controlador executará o *script*. Assim, percebe-se a possibilidade de reuso dos *scripts* para diferentes dados de teste.

Caso ocorra um erro durante a execução do *script*, a classe *Controlador* irá chamar a classe *ControladorErro* para o devido tratamento. Esta classe escolherá o *script* informado no arquivo *Erros.xml*.

4.4 Uso dos Padrões

Temos duas utilizações do *Factory Method* no *framework* *FuncTest*. Na primeira utilização, temos uma *Factory* para criação de *scripts*. Neste caso o *framework* delega à *Factory* a decisão de qual *script* será criado. Na segunda utilização, temos uma *Factory* para geração de um Objeto DAO. Objeto DAO é utilizado para recuperar os dados de teste.

O diagrama de classes abaixo (Figura 4) evidencia o uso do padrão DAO no *framework*.

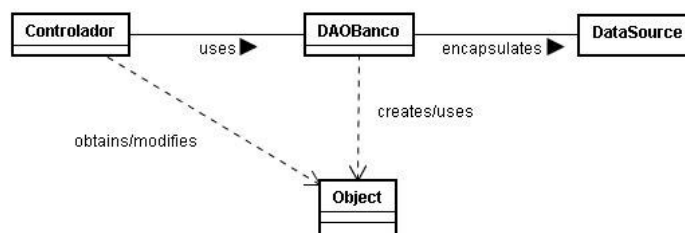


Figura 5 – Uso do Padrão DAO para selecionar a massa de teste

Na tabela 2, é apresentada uma correspondência entre os participantes do padrão DAO e as classes do *framework* que implementam este padrão.

Participantes do Padrão DAO	Participantes Correspondentes no FuncTest
BusinessObject	Controlador
DataAccessObject	DAOBanco
DataSource	DataSource
TransferObject	Object[]

Tabela 1 – Classes do FuncTest que implementam o Padrão DAO

O diagramas de classes abaixo (Figura 5 e Figura 6) evidenciam respectivamente os dois usos do padrão *Factory Method* no *framework*.

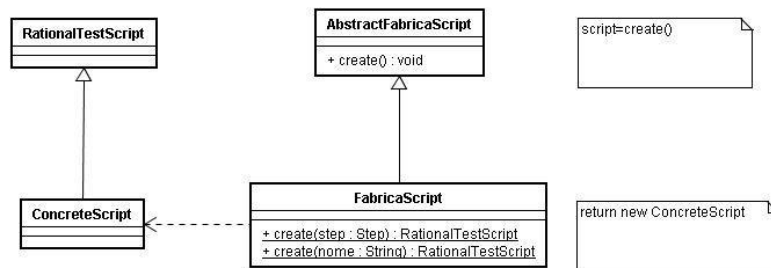


Figura 6 – Uso do Padrão *Factory Method* para a criação de *scripts* de teste

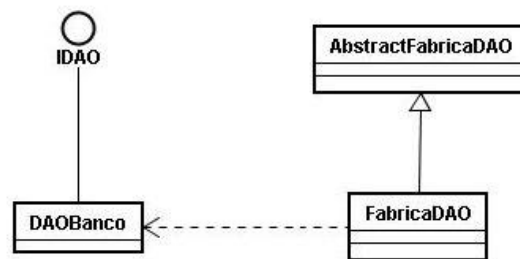


Figura 7 – Uso do Padrão *Factory Method* para a criação dos objetos DAO

Na tabela 2, é apresentada uma correspondência entre os participantes do padrão *Factory Method* e as classes do *framework* que implementam este padrão.

Participantes do Padrão <i>Factory Method</i>	Participantes Correspondentes no FuncTest	
	Fábrica Script	Fábrica DAO
Product	RationalTestScript	IDAO
ConcreteProduct	Script	DAO
Creator	AbstractFabricaScript	AbstractFabricaDAO
ConcreteCreator	FabricaScript	FabricaDAO

Tabela 2 – Classes do *FuncTest* que implementam o Padrão *Factory Method*

5. Conclusão

Scripts gerados a partir de ferramentas *Record and Playback* normalmente são pouco reutilizáveis e pouco manuteníveis. De forma a minorar este problema diversos *frameworks*, utilizando as técnicas *Data-driven* e *Keyword-driven*, tem sido propostos para automatização de testes funcionais.

Neste artigo, foi apresentado um *framework* para a automação de testes sistêmicos funcionais, denominado *FuncTest*, que, além das técnicas tradicionais, utiliza a arquitetura MVC e os padrões *Factory Method* e DAO para aprimorar a manutenibilidade e a reusabilidade de projetos de automação.

O uso do FuncTest, num processo de desenvolvimento aderente ao CMM nível 2, trouxe um aumento significativo da produtividade de novos projetos de automação de testes funcionais e ofereceu, dentre outros, os seguintes benefícios ao projeto de teste:

- desacoplamento, através da arquitetura MVC entre passos do caso de teste e *scripts* a serem executados;
- independência entre *scripts* e dados de teste segundo a técnica *Data-driven*;
- modularização dos *scripts* utilizando a técnica *Keyword-driven* com o controle independente de chamada dos scripts e conseqüente redução do acoplamento entre eles;
- facilidade de inclusão de novos casos de teste nos procedimentos de teste;
- independência em relação ao SGBD utilizado obtida através do padrão DAO;
- controle de erros centralizado;
- melhoria no tempo de execução de testes uma vez que scripts de tratamento de erro são invocados automaticamente reduzindo as paradas por exceções;
- configuração do framework através de um conjunto de arquivos XML só sendo necessário manipular código java na elaboração dos scripts;
- melhoria da reusabilidade e da manutenibilidade de scripts;
- melhor legibilidade de código.

Como trabalhos futuros, pretendemos utilizar o padrão Strategy no tratamento de erros ocorridos durante a automação. Além disso, pretendemos adaptar o framework para a utilização da técnica model-based testing [9], possibilitando a geração automática de testes.

6. Referências Bibliográficas

- [1] MYERS, Glenford J. The Art of Software Testing. New York: John Wiley & Sons, Second Edition, 2004.
- [2] FEWSTER, Mark, GRAHAM, Dorothy. Software Test Automation. Addison-Wesley Professional; 1st edition, 1999.
- [3] DUSTIN, Elfriede. Effective Software Testing: 50 Specific Ways to Improve Your Testing. Addison-Wesley Professional; 1st edition, 2002.
- [4] DUSTIN, Elfriede, RASHKA, Jeff, PAUL, John. Automated Software Testing: Introduction, Management, and Performance. Addison-Wesley Professional; Bk&CD Rom edition, 1999.
- [5] GAMMA, E., HELM, R., JOHNSON, R., VLISSIDES, J. Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1995.

- [6] BUSCHMANN, Frank, MEUNIER, Regine, ROHNERT, Hans, SOMMERLAD, Peter, STAL, Michael. Pattern-Oriented Software Architecture: A System of Patterns. New York: John Wiley & Sons, 1996.
- [7] SCHMIDT, Douglas, STAL, Michael, ROHNERT, Hans, BUSCHMANN, Frank. Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects, Volume 2. October 2000.
- [8] ALUR, Deepak, MALKS, Dan, CRUPI, John. Core J2EE Patterns: Best Practices and Design Strategies, 2 Ed. California: Sun Microsystems, 2003.
- [9] DALAL, S. R., JAIN, A., KARUNANITHI, N., BELLCORE, N., LEATON, J. M., LOTT, C. M., PATTON, G. C., HOROWITZ, B. M. Model-Based Testing in Practice. International Conference on Software Engineering, 1999.
- [10] FANTINATO, Marcelo, et al. AutoTest – Um Framework Reutilizável para a Automação de Teste Funcional de Software. Simpósio Brasileiro de Qualidade de Software, 2004.
- [11] Framework automation with IBM Rational Functional Tester: Data-driven. Disponível em http://www-128.ibm.com/developerworks/rational/library/05/1108_kelly/.
- [12] Framework automation with IBM Rational Functional Tester: Keyword-driven. Disponível em http://www-128.ibm.com/developerworks/rational/library/06/0523_kelly/.